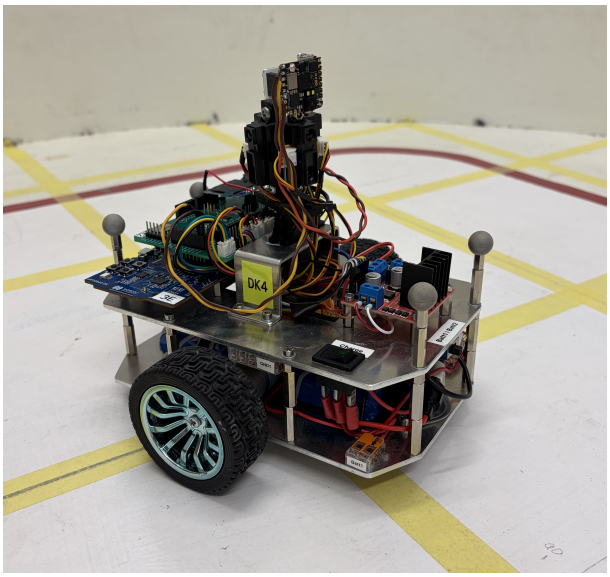
NTNU

Department of Engineering Cybernetics

TTK8 - Construction of embedded systems

Investigation and Mitigation of UART Communication Interference between the Nicla Vision and a SLAM Robot



Reitan, Martin

martigre@stud.ntnu.no

Autumn 2026

Summary

This project investigated the reliability of the Universal Asynchronous Receiver-Transmitter (UART) communication between the Nicla Vision camera and the Norwegian University of Science and Technology (NTNU) Simultaneous Localization and Mapping (SLAM) robot. A test framework was developed to monitor communication performance by tracking received packets, timeouts, UART errors, and missing or suspicious sequence numbers. A total of 40 tests were performed to investigate the effects of baud rate, camera processing, servo activity, and cable configuration. The results showed that camera image processing had the greatest impact on communication reliability, as all tests with image processing off achieved a 0% failure rate. With image processing on, 57600 baud provided the lowest failure rate. Servo activity had no significant effect, while changing the cable configuration generally reduced reliability. Several potential improvements were identified, including adjusting the baud rate and timeout or moving image processing to the robot platform. However, no final improvements were implemented. The results will provide a basis for further development and testing of a more reliable communication system.

Table of Contents

List of Figures **iv**

List of Tables **iv**

List of Acronyms **v**

1 Background **1**

2 Success Criteria **1**

3 System Architecture **2**

3.1 Communication Protocol 3

3.2 Potential Sources of Communication Errors 4

4 Test Setup **5**

4.1 Hardware connections 5

4.2 Necessary software 5

4.3 Test Procedure 7

4.4 Test Spreadsheet 9

5 Test Results **10**

6 Analyzing the UART Reliability Tests **11**

6.1 Errors introduced with the test framework 13

7 Improvements **13**

7.1 Baud Rate 13

7.2 Timeout Configuration 14

7.3 Moving the Camera Processing to the Robot 14

7.4 Conclusion 15

References **16**

A Nicla Vision Code (.py) **17**

B Robot Header File (.h) **22**

C Robot C File (.c) **23**

D Work Breakdown Structure **31**

E Plan of Progress **32**

F AI Declaration **33**

List of Figures

1 Overview of the Context diagram for the UART communication system. 3

2 Overview of the Inner Analysis for the UART communication system. 3

3 Nicla Vision with its connected cables for power and UART connection. 6

4 Normal/twisted UART cable setup. 8

5 Changed/separated UART cable setup. 8

6 Debug terminal. 11

7 UART test printed to debug terminal. 11

List of Tables

1 UART pin connection setup. 5

2 Summary of UART reliability tests. 10

List of Acronyms

IDE Integrated Development Environment. 5, 6

LiPo Lithium-ion polymer. 5

NTNU Norwegian University of Science and Technology. i, 1

SLAM Simultaneous Localization and Mapping. i, 1, 2, 5, 13–15

UART Universal Asynchronous Receiver-Transmitter. i, 1, 2, 4–15

1 Background

The SLAM robot project at NTNU has been extended with a camera module to improve its ability to detect and characterize obstacles in its surroundings. The camera is an addition to the robot's existing sensors by providing information about obstacle position, width, and distance. This information can improve the robot's navigation and environment-mapping capabilities.

Communication between the camera and the robot has been implemented using UART communication. The robot sends a request to the camera, which returns processed sensor data containing the calculated measurements. For the system to operate reliably, the transmitted data must be received correctly, consistently, and within an acceptable time.

Previous testing has revealed problems with the UART communication, including timeouts and missing packets. These issues can cause the robot to use invalid camera measurements. The underlying cause of the observed communication problems has not yet been established [2].

Reliable communication is important for the overall performance of the SLAM system. Incorrect or missing camera measurements may reduce the usefulness of the camera module and make the system less robust. This project therefore investigates the reliability of the existing UART connection, identifies conditions that contribute to communication failures, and evaluates potential improvements.

2 Success Criteria

The objective of this project is to characterize and improve the reliability of UART communication between the Nicla Vision camera and the SLAM robot. The project should present a reproducible testing method, document the observed behavior, and evaluate whether the considered improvements could reduce communication failures.

The following deliverables are defined:

- **Test framework:** Develop code for systematically testing the UART connection and log relevant data, including the number of requests, successfully received packets, timeouts, communication errors, and missing sequence numbers.
- **Setup instructions:** Document the hardware connections, software configuration, test procedure, and parameters required to reproduce the experiments.
- **Test results:** Perform controlled experiments, present the results in tables and figures, and analyze the relationship between the tested conditions and communication reliability.
- **Improvements:** Implement or recommend changes to the communication protocol, software, or hardware setup, and evaluate their potential effectiveness against the original implementation.

The success of the project will be determined by comparing the communication performance before and after potential improvements. Relevant performance indicators include the timeout rate, error rate, and the number of missing or invalid packets.

3 System Architecture

Based on the environmental model, the environment consists of the SLAM robot, the Nicla Vision camera module, and the Tester as terminators. While the system is the UART communication interface connecting them. The robot uses an nRF microcontroller to manage communication and other actuators, while the camera runs its image-processing program on the Nicla Vision platform. The tester is responsible for providing test parameters and analyzing the final status and errors. The Context diagram can be seen in Figure 1. An Inner Analysis of the UART communication system, visualizing the different main components implemented in this project and from earlier in the system, can be seen in Figure 2.

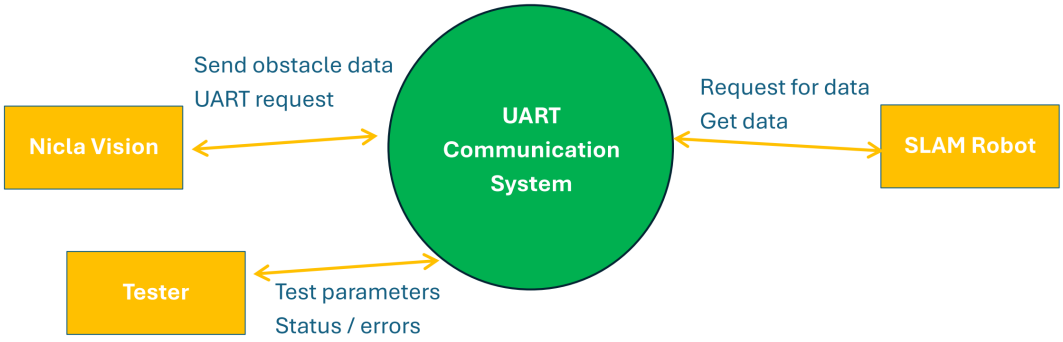


Figure 1: Overview of the Context diagram for the UART communication system.

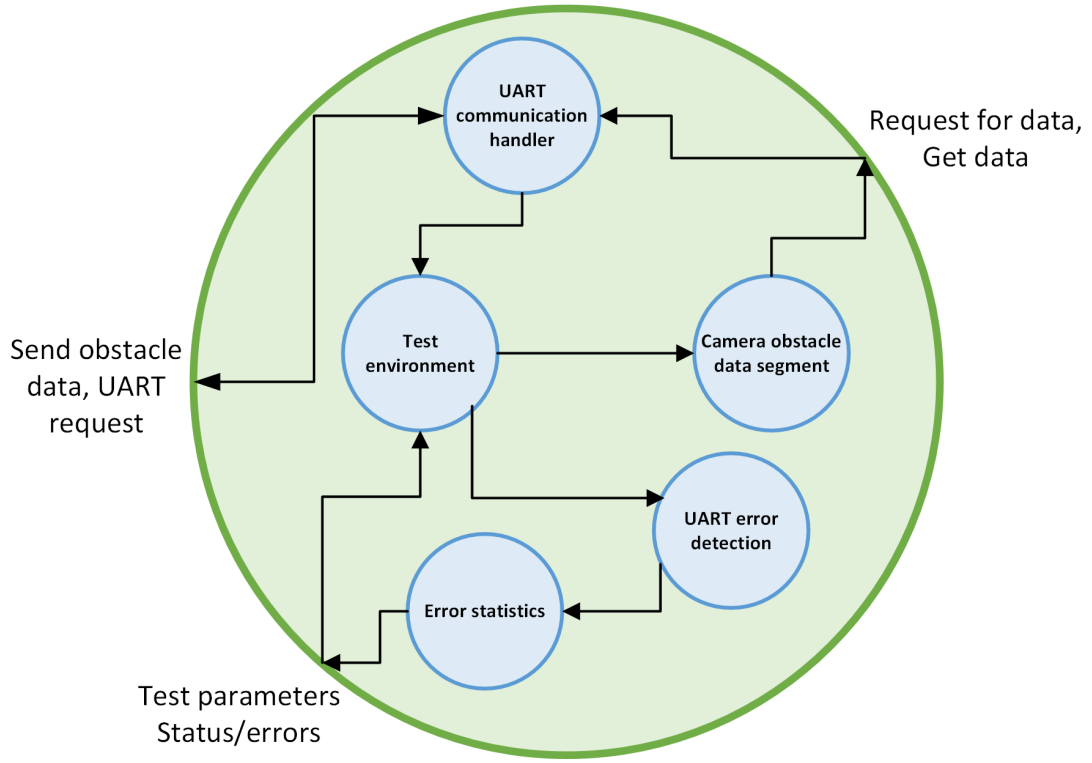


Figure 2: Overview of the Inner Analysis for the UART communication system.

3.1 Communication Protocol

The communication follows a request-response structure. The robot transmits a request, and the camera responds with a packet containing the current measurement. The response is encoded as binary data, with each measurement represented by a fixed number of bytes.

A sequence number can be included in the packet to help identify missing, duplicated, or unexpected measurements [3]. By comparing the following sequence

numbers, the receiver (robot) can detect when one or more packets may have been skipped. However, sequence numbers alone cannot detect every form of data corruption.

3.2 Potential Sources of Communication Errors

There are several factors that can contribute to the observed communication problems. These should be treated as hypotheses until they have been tested.

- **Timing and timeouts:** The camera requires time to process images and calculate measurements. If the response from the camera takes longer than the configured timeout from the robot side, the robot could leave the receive operation, causing a timeout.
- **UART configuration:** Incompatible baud rates or other communication settings may prevent reliable data reception.
- **Processing interference:** The processing of images on the camera or the robot controlling the servo tower, may interfere with transmit and receive operations.
- **Electrical interference:** Noise, grounding problems, wiring, or signal integrity may affect the transmitted data.
- **Software handling:** Incorrect handling of asynchronous transmit and receive operations, aborted transfers, or delayed responses may result in missing or incorrectly interpreted packets.

The test framework for this project has been designed to distinguish these hypotheses and identify which changes have a measurable effect on communication performance.

4 Test Setup

The test setup consists of the SLAM robot and the Nicla Vision camera connected through their UART connection. The robot sends requests to the camera and records the outcome of each communication attempt. The camera executes its obstacle-detection program and returns the corresponding measurement packet.

The test results include the number of requests sent, successfully received packets, timeouts, UART errors, and missing and suspicious sequence numbers, all recorded by being printed to a debug terminal.

4.1 Hardware connections

The Nicla Vision and the nRF52840-DK on the robot are both connected to a common ground. Both are powered by a Lithium-ion polymer (LiPo) battery through a voltage regulator. The UART connection between the Nicla Vision and the nRF uses the following pin connections:

Nicla Vision Pin	**nRF52840-DK Pin**
Pin 2 (RX)	P0.14 (TX)
Pin 1 (TX)	P0.12 (RX)

Table 1: UART pin connection setup.

Overviews of the different pinouts can be found on the components main pages at Nordic semiconductor [6] and Arduino [4]. For reading the debug terminal, a USB cable is needed to connect the nRF to a computer.

4.2 Necessary software

The test setup also consists of a software part. The Nicla Vision is programmed in python with the use of the OpenMV Integrated Development Environment (IDE). The program has been used to implement changes to the test framework of the

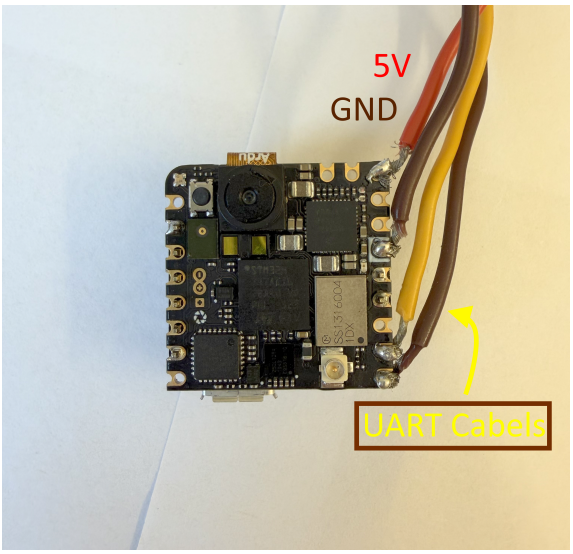


Figure 3: Nicla Vision with its connected cables for power and UART connection.

original obstacle-detection program made by Alve in her specialization project [1], as well as for debugging. The main changes are the addition of an incrementing sequence number and the option to turn off the camera processing, so that only static values are sent from the camera. The updated code can be found in appendix A.

The nRF52840-DK is programmed in C with the use of the SEGGER Embedded Studio IDE. The test framework has been implemented into the C-programs created by Alve that handle the UART communication between the camera and nRF [1]. The test framework includes multiple error detections with counters. These errors are timeouts, general UART errors, and with the use of the sequence number, it is possible to detect packets that are skipped or duplicated. Also, if there is something wrong with the sequence number, it could mean that the whole packet is corrupted. In the end of a test, the program will print out a log of how many times each error has occurred. The updated code can be found in appendix B and C.

The other relevant robot code can be found in the GitHub repository *robot-code* located in the GitHub organization *SLAMRobotProject* [7]. In the file *task_sensor_tower.c*, can the servo tower be turned off.

4.3 Test Procedure

Before performing each test, it was ensured that the same initial setup and a clearly defined set of test parameters were used. It is important to change only one test parameter at a time when comparing configurations. This makes it easier to characterize the differences in communication performance based on the one parameter being investigated.

The test procedure in the project was as follows:

1. Open the relevant codes found in appendices A, B, and C.
2. Configure the robot and camera with matching UART settings or software/hardware changes (test parameters). Remember to set `UART_test = true` in the robot code (appendix C).
3. Download the relevant software codes for the camera and robot. Downloading the camera program first is recommended. Keep the USB cable connected to the nRF52840-DK.
4. Turn on the robot battery power and start the debug terminal in SEGGER Embedded Studio.
5. Let the program run for the predefined number of requests.
6. Record the communication statistics published in the terminal and the test parameter changes.
7. Repeat the test for other test parameters.
8. Compare the results.

The test variables in this project included baud rate, camera processing activity, servo activity, cable coiling, and software changes. These variables were tested separately from each other. For each test, the current test variable/parameter was noted, together with the setup.

The possible states for the test variables were as follows:

- **Baud rate:** 115200 / 57600 / 38400 / 19200 / 9600
- **Camera:** ON / OFF
- **Servo:** ON / OFF
- **Cable:** Normal / Changed (see figures 4 and 5)

The original setup of variables was as follows:

- **Baud rate:** 115200
- **Camera:** ON
- **Servo:** ON
- **Cable:** Normal (see Figure 4)
- **UART timeout:** 250 ms

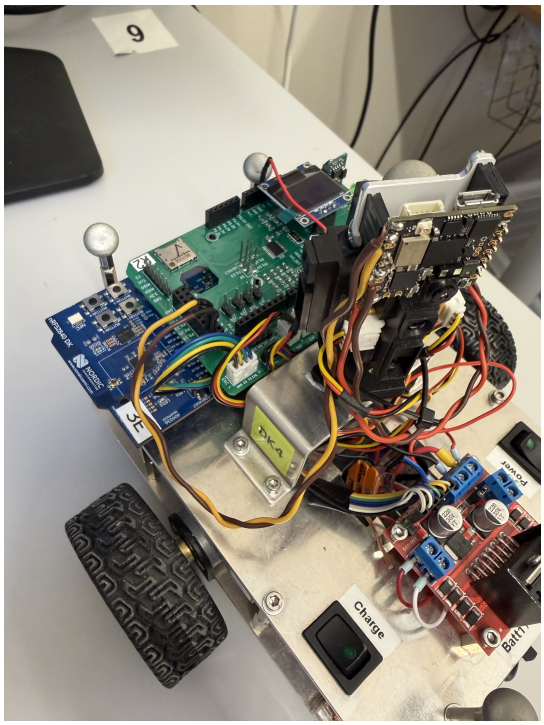


Figure 4: Normal/twisted UART cable setup.

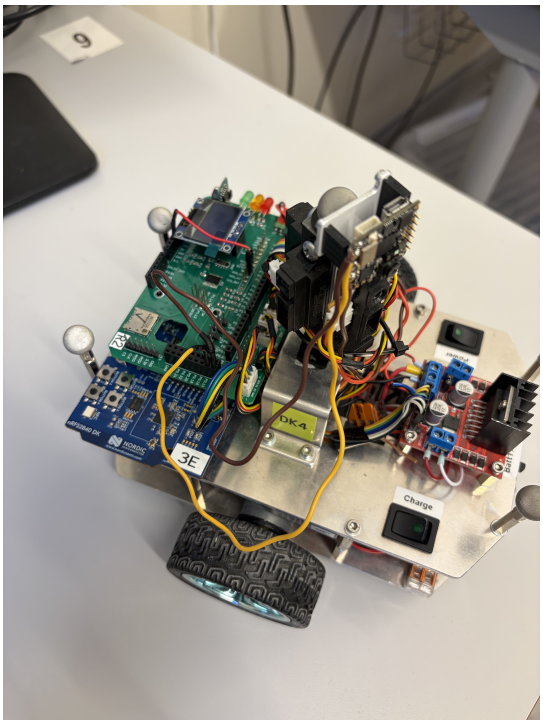


Figure 5: Changed/separated UART cable setup.

4.4 Test Spreadsheet

An Excel spreadsheet was set up to document the different statistical values printed to the debug terminal, as well as the variable setup. A table was used to make the test results more organized and easy to analyze. The following table columns were defined:

Test nr., ***Baud rate***, ***Camera***, ***Servo***, ***Cable***, ***Requests***, ***Received***, ***Time-outs***, ***UART errors***, ***Missing seq.***, ***Suspicious seq.***, ***Not received***, ***Failure rate***.

Failure rate was calculated as:

$$R_{\text{failure}} = \frac{N_{\text{received}}}{N_{\text{requests}}} \times 100\%$$

where N_{received} is the number of received responses, and N_{requests} is the total number of requests sent. The rates of the other types of errors could also be calculated in the same way by changing out the N_{received}, with the other numbers.

5 Test Results

A total of 40 tests were conducted with different combinations of test variables. Every test ends after a total of 1000 requests/responses of packets have been completed. Test number 1 was conducted on the original setup of the system. A full summary of the UART reliability tests is presented in Table 2. Test number 35, after multiple attempts, was unable to complete 1000 requests/responses.

Test nr.	Baud rate	Camera	Servo	Cable	Requests	Received	Timeouts	UART errors	Missing seq.	Suspicious seq.	Not Received	Failure rate
1	115200	ON	ON	Normal	1000	964	109	182	239	122	36	3,6 %
2	57600	ON	ON	Normal	1000	989	46	40	237	189	11	1,1 %
3	38400	ON	ON	Normal	1000	899	120	12	80	9	101	10,1 %
4	19200	ON	ON	Normal	1000	912	107	30	120	41	88	8,8 %
5	9600	ON	ON	Normal	1000	834	193	42	100	314	166	16,6 %
6	115200	OFF	ON	Normal	1000	1000	0	0	0	0	0	0,0 %
7	57600	OFF	ON	Normal	1000	1000	0	0	0	0	0	0,0 %
8	38400	OFF	ON	Normal	1000	1000	0	0	0	0	0	0,0 %
9	19200	OFF	ON	Normal	1000	1000	0	0	0	0	0	0,0 %
10	9600	OFF	ON	Normal	1000	1000	0	0	0	0	0	0,0 %
11	115200	ON	OFF	Normal	1000	980	87	138	263	202	20	2,0 %
12	57600	ON	OFF	Normal	1000	982	45	44	236	186	18	1,8 %
13	38400	ON	OFF	Normal	1000	927	113	25	208	119	73	7,3 %
14	19200	ON	OFF	Normal	1000	885	155	45	185	58	115	11,5 %
15	9600	ON	OFF	Normal	1000	851	180	37	143	275	149	14,9 %
16	115200	OFF	OFF	Normal	1000	1000	0	0	0	0	0	0,0 %
17	57600	OFF	OFF	Normal	1000	1000	0	0	0	0	0	0,0 %
18	38400	OFF	OFF	Normal	1000	1000	0	0	0	0	0	0,0 %
19	19200	OFF	OFF	Normal	1000	1000	0	0	0	0	0	0,0 %
20	9600	OFF	OFF	Normal	1000	1000	0	0	0	0	0	0,0 %
21	115200	ON	ON	Changed	1000	875	113	228	359	238	125	12,5 %
22	57600	ON	ON	Changed	1000	934	58	62	320	258	66	6,6 %
23	38400	ON	ON	Changed	1000	846	116	66	230	138	154	15,4 %
24	19200	ON	ON	Changed	1000	825	125	75	163	72	175	17,5 %
25	9600	ON	ON	Changed	1000	814	210	122	145	40	186	18,6 %
26	115200	OFF	ON	Changed	1000	1000	0	0	0	0	0	0,0 %
27	57600	OFF	ON	Changed	1000	1000	0	0	0	0	0	0,0 %
28	38400	OFF	ON	Changed	1000	1000	0	0	0	0	0	0,0 %
29	19200	OFF	ON	Changed	1000	1000	0	0	0	0	0	0,0 %
30	9600	OFF	ON	Changed	1000	1000	0	0	0	0	0	0,0 %
31	115200	ON	OFF	Changed	1000	886	96	209	367	254	114	11,4 %
32	57600	ON	OFF	Changed	1000	954	40	56	122	78	46	4,6 %
33	38400	ON	OFF	Changed	1000	806	145	81	110	15	194	19,4 %
34	19200	ON	OFF	Changed	1000	811	137	66	144	46	189	18,9 %
35	9600	ON	OFF	Changed	1000	NaN	NaN	NaN	NaN	NaN	NaN	NaN
36	115200	OFF	OFF	Changed	1000	1000	0	0	0	0	0	0,0 %
37	57600	OFF	OFF	Changed	1000	1000	0	0	0	0	0	0,0 %
38	38400	OFF	OFF	Changed	1000	1000	0	0	0	0	0	0,0 %
39	19200	OFF	OFF	Changed	1000	1000	0	0	0	0	0	0,0 %
40	9600	OFF	OFF	Changed	1000	1000	0	0	0	0	0	0,0 %

Table 2: Summary of UART reliability tests.

```
<info> app: seq: 909, x_start_mm: -264, x_width_mm: 572, distance_mm: 837
<warning> app: Sequence gap: previous=909 received=911 missed=1
<info> app: Camera payload 7 bytes
<info> app: seq: 911, x_start_mm: -243, x_width_mm: 509, distance_mm: 771
<info> app: Camera payload 7 bytes
<info> app: seq: 912, x_start_mm: -205, x_width_mm: 410, distance_mm: 638
<warning> app: UART RX timeout, no connection to camera
<info> app: Camera payload 7 bytes
<info> app: seq: 0, x_start_mm: 0, x_width_mm: 0, distance_mm: 0
<info> app: Camera payload 7 bytes
<info> app: seq: 913, x_start_mm: -240, x_width_mm: 425, distance_mm: 576
<error> app: UART ERROR: mask = 0x00000001
```

Figure 6: Debug terminal.

```
Debug Terminal
<info> app: ========== UART TEST ==========
<info> app: Packets requested: 900
<info> app: Packets received:  798
<info> app: Timeouts:          85
<info> app: UART errors:       245
<info> app: Missing packets    345
<info> app: Suspicious seq.    248
<info> app: Not received:      102
<info> app: ================================
```

Figure 7: UART test printed to debug terminal.

Figure 6 shows what can be found in the debug terminal. From line 1 to line 4 of the terminal, one can see a missing sequence number. Line 7 has the error message for timeout, and the last line has a UART error message. Figure 7 shows how the printout of the different counters looks. The values can be used further in spreadsheets.

It is worth noting that redoing the tests can produce different numbers, but the characteristics of the values should remain the same.

6 Analyzing the UART Reliability Tests

By analyzing Table 2, we can see the different effects the test variables have on reliability. Looking at the five first tests, we can see that 57600 baud produced the

lowest failure rate of 1.1%, compared to 3.6% at 115200 baud. However, reducing the baud rate does not necessarily improve reliability. At 9600 baud, the failure rate increased to 16.6%. This indicates that the relationship between baud rate and communication reliability is not necessarily linear and that other factors may influence the results. This characteristic can also be seen in the other sets of baud rate tests. Looking at the other error counters, such as timeouts, we can also see that the baud rate of 57600 baud performs better than 115200 baud. From looking only at the baud rates we could conclude that the current system with baud rate of 115200 baud, could benefit from changing the baud rate to 57600 baud. It is a sign that the current system needs more time before sending data over the UART communication.

Looking at the effect of having image processing on the Nicla Vision camera or not has shown significant differences in reliability. Every test where the camera processing was off and only sent fixed values in addition to the sequence number had a failure rate of 0%. With this observation, it is very clear that the image processing happening locally on the Nicla Vision is a significant cause of the problems with communication and missing packets.

The effect of having a rotating servo tower in the robot program, does not seem to have any major effect on the reliability of the UART communication. Specifically when comparing to the effect of the camera and baud rate. The cable setup *Changed* seems to have a worse failure rate compared to the original setup of twisted cables. The reason for this is unknown, but it could be other factors such as different voltage levels left in the battery, this is relevant for every test performed.

When every variable is taken into consideration we can see that the cable setup should remain as in Figure 4, the rotating servo can still be rotating, the image processing should be moved somewhere else then locally on the Nicla Vision, and the baud rate could be lowered down to 57600 baud.

6.1 Errors introduced with the test framework

When comparing the errors from the original code without the test framework to those from the code with the test framework, it was discovered that the introduction of a sequence number also introduced some extra problems, such as missing and suspicious sequence numbers. To include the sequence number, the data length had to be increased to 8 data bits instead of the original 6 data bits. Longer data packets can cause more errors. So the final code will not include the test framework for normal use in the SLAM project. The introduction of sequence numbers has helped identify the UART errors, so they have not been useless in the test setup.

7 Improvements

Several improvements to the UART communication between the SLAM robot and the Nicla Vision camera module have been considered based on the analysis of the test results. The most relevant ones will be described in the following subsections and how they could affect the system compared to the original configuration. Though none of the improvements have yet been implemented in the existing SLAM setup.

7.1 Baud Rate

One possible improvement is to change the baud rate used for communication between the robot and the camera module. The original setup uses a baud rate of 115200 baud. As discussed in section 6, the test results show that the UART reliability varies depending on the selected baud rate.

A lower baud rate can provide greater timing margins and reduce the sensitivity to clock inaccuracies between the transmitting and receiving devices [5]. However, it also increases the time required to transmit each packet. Based on the test results, 57600 baud could be a better alternative to the original 115200 baud setup. Further testing would be required to determine whether this improvement is consistent under different operating conditions.

7.2 Timeout Configuration

Another possible improvement is to adjust the timeout used by the robot when waiting for data from the camera. The timeout determines how long the robot waits for a response before considering the communication as unsuccessful.

Increasing the timeout could reduce the number of reported timeouts and lost packets if the camera needs more time to process an image and generate a UART response. However, a longer timeout also increases the time the robot could spend waiting for a response that potentially never arrives. Ultimately, this could affect the update rate of the camera measurements and the overall responsiveness of the SLAM system, causing more harm than originally intended.

The timeout should be selected based on the measured response time of the camera, including its processing time and communication delay. A systematic test of different values for timeout could also be done to find a satisfactory timeout. Either way, the timeout should provide sufficient margin for normal operation without delaying the robot too much when a response is missing.

Some small tests involving changes to the timeout have been attempted to observe any of the expected changes. Increasing the timeout showed promising results in terms of fewer timeouts and better reliability, but more tests are needed.

7.3 Moving the Camera Processing to the Robot

A more extensive improvement would be to move the camera processing from the Nicla Vision module to the robot. In the current configuration, the camera handles the image processing and sends the resulting measurements of objects to the robot through UART. This introduces the possibility of packet loss, timeouts, and communication errors, as seen in the test results.

Moving the processing to the robot could eliminate the need to do any type of processing on the camera itself, and rather only send the image data. With this setup, UART might be changed for some other type of communication to transmit

the image.

However, this approach would require the nRF52840-DK to support the camera processing algorithms and the necessary hardware changes. The current camera implementation utilizes the OpenMV libraries and functionality that are designed for the Nicla Vision platform. Reimplementing these functions on the nRF could therefore require a considerable amount of work. It would also be worth considering whether the nRF could perform the image processing without affecting the other SLAM tasks.

In the end, moving the image processing to the robot is a possible long-term improvement, but it may not be the most practical solution for the current system. Other options might be considered first.

7.4 Conclusion

The test results indicate that changing the baud rate is a simple improvement worth investigating further. In particular, 57600 baud resulted in the lowest failure rate with the preferred configuration of having both the camera and servo enabled and the normal cable configuration. Adjusting the timeout could also improve the handling of delayed responses caused by image processing on the camera. Moving the camera processing to the robot could eliminate a lot of the UART errors, as seen in the results, but it would require significant software and hardware changes.

No final improvement was implemented during this project. The proposed changes should therefore be considered as further work in either a specialization project or a master's thesis. Further testing and implementation would be necessary to determine which combination provides the most reliable communication without negatively affecting the performance of the SLAM system.

References

[1] Rebekka Alve. "Nicla Vision Camera on SLAM Robot". Specialization Project in Cybernetics and Robotics. Norwegian University of Science and Technology, Dec. 2025.

[2] Rebekka Alve. "Power Redesign and Camera Rotation on SLAM Robot". Master's thesis in Cybernetics and Robotics. Norwegian University of Science and Technology, June 2025.

[3] Massachusetts Institute of Technology. *Reliable Data Transport Protocols*. Tech. rep. 6.02 Introduction to EECS II: Digital Communication Systems, Chapter 19, Section 19.2.1. MIT OpenCourseWare, 2012. URL: https://ocw.mit.edu/courses/6-02-introduction-to-eecs-ii-digital-communication-systems-fall-2012/2d3a62fd8f18f6bf69c70bc73f7d6ca9_MIT6_02F12_chap19.pdf.

[4] Christopher Méndez. *Nicla Vision User Manual*. URL: https://docs.arduino.cc/tutorials/nicla-vision/user-manual/ (visited on Oct. 21, 2026).

[5] Microchip Technology. *Debugging Serial Interfaces in Embedded Systems*. Tech. rep. TB3331. Microchip Technology, 2023. URL: https://www.microchip.com/content/dam/mchp/documents/MCU08/ApplicationNotes/ApplicationNotes/TB3331-Debug-SerialInterf-EmbeddedSystems-DS90003331.pdf?.

[6] Nordic Semiconductor. *nRF52840 DK Development Kit*. URL: https://www.nordicsemi.com/Products/Development-hardware/nRF52840-DK (visited on Oct. 21, 2026).

[7] TOnshus, runemell, et al. *SLAMRobotProject*. URL: https://github.com/SLAMRobotProject (visited on Oct. 25, 2026).

A Nicla Vision Code (.py)

```python
import sensor, time, struct
from pyb import UART, LED
from machine import I2C
from vl53l1x import VL53L1X

# Enable UART4, Tx = pin1 (SDA i2c), Rx = pin2 (SCL i2c)
uart = UART(4, 115200, timeout_char=200)

# Parameters BLOB DETECTION
OFFSET = 30  # threshold offset around mean background brightness
min_area = 300       # ignore tiny blobs
min_pixels = 300     # ignore tiny blobs
max_fraction = 0.95   # ignore blobs covering more than 90% of image (not working?)

# Camera setup
sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)  # grayscale for detection
sensor.set_framesize(sensor.QVGA) # smaller frame size for speed
sensor.skip_frames(time=500)

# Camera specifications and focal length
pixel_size_mm = 1.75e-3
sensor_px_width = 1616      # active pixel array width
image_px_width  = 320       # QVGA capture width
f_mm = 2.2                  # focal length in mm
f_tuned_mm = 100.0            # tuned focal length, increase value to decrease object width estimates

sensor_width_mm = pixel_size_mm * sensor_px_width
focal_length_px = f_tuned_mm + int((f_mm * image_px_width) / sensor_width_mm) # approx 248 + tuned value

# Distance sensor setup (ToF = time of flight)
i2c = I2C(2)
tof = VL53L1X(i2c)
MIN_VALID_DISTANCE = 40  # mm
```

```
MAX_VALID_DISTANCE = 2000 # mm

# Buffer (last 10 readings) for smoothing noisy detections
readings = []
MAX_READINGS = 10

# LEDs
red = LED(1)
green = LED(2) # streaming video
blue = LED(3)  # setting up stream

def leds_off():
    red.off()
    green.off()
    blue.off()

clock = time.clock()

def uart_request():
    if uart.any():
        request = uart.read(1)
        print('Request:', request)
        if request == b'r':
            return True
    return False

def average_offset_width_distance_mm(target,center_x,dist):

    # Get x-range of the blob, NB: pixels 0-320
    x_min = target.x
    x_max = target.x + target.w

    # Store in buffer
    readings.append((x_min,x_max,dist))

    # Find average of the last N readings
    if len(readings) == MAX_READINGS:
        x_min = sum(r[0] for r in readings) // MAX_READINGS
        x_max = sum(r[1] for r in readings) // MAX_READINGS
```

```
74          dist = sum(r[2] for r in readings) // MAX_READINGS
75          readings.pop(0)  # remove oldest reading
76
77      # Convert from pixel to mm
78      width_px = x_max - x_min
79      object_width_mm = (width_px * dist) / focal_length_px
80
81      # offset from image center to x_start in mm (negative = left)
82      x_offset_mm = ((x_min - center_x) * dist) / focal_length_px
83
84      return int(x_offset_mm),int(object_width_mm),int(dist)
85
86  def detect_obstacles():
87      clock.tick()
88      img = sensor.snapshot()
89
90      ## DISTANCE SENSOR
91      dist = tof.read()  # Read distance in mm
92
93      # Dynamic background brightness
94      stats = img.get_statistics()
95      mean_val = stats.l_mean
96      limit = max(0, mean_val - OFFSET)
97      color_thresholds = [(0, limit)]
98
99      # Find dark blobs
100     blobs = img.find_blobs(color_thresholds, pixels_threshold=
    min_pixels, area_threshold=min_area)
101
102     if blobs and dist > MIN_VALID_DISTANCE and dist <
    MAX_VALID_DISTANCE:
103         # Center of the image (where ToF points)
104         center_x = img.width() // 2
105         center_y = img.height() // 2
106
107         # Filter blobs to ignore very large ones
108         max_pixels = int(img.width() * img.height() * max_fraction)
109         valid_blobs = [b for b in blobs if b.pixels < max_pixels]
110
```

```
111         # Find the blob covering the distance sensor's line-of-
    sight (center)
112         target = None
113         for b in valid_blobs:
114             if b.pixels < (img.width() * img.height() *
    max_fraction):
115                 if b.x <= center_x <= b.x+b.w and b.y <= center_y
    <= b.y+b.h:
116                     target = b
117                     break
118         if not target:
119             # If no blob covers the center, find the nearest blob
120             nearest = None
121             nearest_dist = float('inf')
122             for b in blobs:
123                 # Calculate distance from blob center to image
    center
124                 dx = b.cx - center_x
125                 dy = b.cy - center_y
126                 distance = (dx*dx + dy*dy)**0.5
127                 if distance < nearest_dist:
128                     nearest = b
129                     nearest_dist = distance
130             target = nearest
131
132         if target:
133             # Draw target square and distance
134             img.draw_cross((target.cx, target.cy), color=(0, 255,
    0))
135             img.draw_rectangle(target.rect, color=(0, 0, 0),
    thickness=3)
136             img.draw_string((10, 10), f Distance: {dist} mm , color
    =(255, 255, 255),scale=1.5)
137
138             # Calculate average object offset, width and distance
139             return average_offset_width_distance_mm(target,center_x
    ,dist)
140      return 0,0,0
141
```

```
142
143 camera_on = True # Set to False to disable camera processing
144 sequence = 0
145 try:
146     green.on()
147     print( focal length px: , focal_length_px)
148     print( x offset mm, width mm, distance mm )
149     while True:
150         if camera_on:
151             x_offset, width, distance = detect_obstacles()
152         else:
153             x_offset, width, distance = 50, 60, 70
154
155         data = struct.pack(
156             '<Hhhh',
157             sequence,
158             x_offset,
159             width,
160             distance
161         ) # 2 bytes each
162
163         if(uart_request()):
164             uart.write(data)
165             #print('Data sent')
166             print(
167                  SEQ: , sequence,
168                  X: , x_offset,
169                  W: , width,
170                  D: , distance
171             ) # print values, for testing
172
173             sequence = (sequence + 1) % 65536
174
175         time.sleep_ms(100)
176
177 except Exception as e:
178     leds_off()
179     print( \nProgram finished\n )
180     print( Error: , e)
```

B Robot Header File (.h)

```h
#ifndef UARTE_NICLA_VISION_H
#define UARTE_NICLA_VISION_H

#include <sdk_config.h>
#include  nrfx.h
#include  nrfx_uarte.h
#include  nrf_delay.h
#include  nrf_log.h
#include  nrf_log_ctrl.h
#include  nrf_log_default_backends.h
#include  robot_config.h
#include  thread_mqttsn.h
#include  task.h

// UARTE
#define P1 32
#define UARTE_TX_PIN (14)  // P0.14
#define UARTE_RX_PIN (12)  // P0.12
#define UARTE_BAUDRATE NRF_UARTE_BAUDRATE_9600
#define RX_DATA_LENGTH 8 // Was 6, 8 for testing
#define UARTE_TIMEOUT_MS 250
#define UART_TEST_REQUEST_LIMIT 1000
#define UART_TEST_LOG_INTERVAL 10
#define MAX_PLAUSIBLE_SEQ_GAP 100

typedef struct __attribute__((packed)) {
    int16_t seq;                          // For test
    int16_t x_start_mm;
    int16_t x_width_mm;
    int16_t distance_mm;
} CameraLineEstimate_t;


void task_camera(void *pvParameters);
CameraLineEstimate_t uarte1_get_line_estimate(void); //for
   debugging
void uarte1_init(void);
```

```
37
38 #endif /* UARTE_NICLA_VISION_H */
```

C Robot C File (.c)

```
1  #include  uarte_nicla_vision.h
2
3  static const nrfx_uarte_t uarte1 = NRFX_UARTE_INSTANCE(1);
4  static uint8_t rx_buffer[RX_DATA_LENGTH];
5  static bool rx_done = false;
6  static bool tx_done = false;
7
8  // UART test variables
9  static bool UART_test = true;
10 static uint32_t uart_packets_requested = 0;
11 static uint32_t uart_packets_received = 0;
12 static uint32_t uart_timeouts = 0;
13 static uint32_t uart_errors = 0;
14 static uint32_t uart_missing_packets = 0;
15 static uint32_t uart_suspicious_seq = 0;
16
17 static uint16_t last_seq = 0;
18 static bool first_packet = true;
19
20
21 void uarte1_event_handler(nrfx_uarte_event_t const * p_event, void
       * p_context)
22 {
23     switch (p_event->type)
24     {
25         case NRFX_UARTE_EVT_TX_DONE:
26             tx_done = true;
27             break;
28
29         case NRFX_UARTE_EVT_RX_DONE:
30             rx_done = true;
31             break;
```

```
32
33          case NRFX_UARTE_EVT_ERROR:
34              if (UART_test){
35                  uart_errors++;
36              }
37
38              NRF_LOG_ERROR( UART ERROR: mask = 0x%08x ,p_event->data
    .error.error_mask);
39              NRF_LOG_FLUSH();
40
41              break;
42
43          default:
44              break;
45      }
46 }
47
48 void uarte1_init(void)
49 {
50      nrfx_uarte_config_t  config = NRFX_UARTE_DEFAULT_CONFIG;
51      config.pseltxd = UARTE_TX_PIN;
52      config.pselrxd = UARTE_RX_PIN;
53      config.baudrate = UARTE_BAUDRATE;
54      config.hwfc = NRF_UARTE_HWFC_DISABLED;
55
56      APP_ERROR_CHECK(nrfx_uarte_init(&uarte1, &config,
    uarte1_event_handler));
57
58      NRF_LOG_INFO( UARTE1 initialized );
59      NRF_LOG_INFO( Baudrate: %d , UARTE_BAUDRATE);
60      NRF_LOG_INFO( RX packet size: %d bytes , RX_DATA_LENGTH);
61      NRF_LOG_INFO( UART test mode: %s , UART_test ?  ON  :  OFF );
62      NRF_LOG_FLUSH();
63 }
64
65 void uarte1_request_data(void)
66 {
67      // Send request
68      tx_done = false;
```

```
69      uint8_t request_byte = 'r';
70      if (UART_test){
71          uart_packets_requested++;
72      }
73      ret_code_t err = nrfx_uarte_tx(&uarte1, &request_byte, 1);
74
75      if (err != NRF_SUCCESS){
76          if (UART_test){
77              uart_errors++;
78          }
79
80          NRF_LOG_WARNING( Could not start UART TX: %d , err);
81          NRF_LOG_FLUSH();
82      }
83  }
84
85  bool uarte1_receive_data(void)
86  {
87      // Receive data
88      rx_done = false;
89      ret_code_t err = nrfx_uarte_rx(&uarte1, rx_buffer,
        RX_DATA_LENGTH);
90      if (err != NRF_SUCCESS){
91          if (UART_test){
92              uart_errors++;
93          }
94
95          NRF_LOG_WARNING( Could not start UART RX: %d , err);
96          NRF_LOG_FLUSH();
97          return false;
98      }
99
100     // Wait for RX with timeout
101     TickType_t start_time = xTaskGetTickCount();
102     while (!rx_done)
103     {
104         __WFE(); // wait for events
105         TickType_t elapsed_time = xTaskGetTickCount() - start_time;
106         if (elapsed_time >= pdMS_TO_TICKS(UARTE_TIMEOUT_MS)) //
```

```
    approximate 1 ms per loop
107         {
108             nrfx_uarte_rx_abort(&uarte1);
109             nrfx_uarte_tx_abort(&uarte1);
110             if (UART_test){
111                 uart_timeouts++;
112             }
113 
114             NRF_LOG_WARNING( UART RX timeout, no connection to
    camera );
115             NRF_LOG_FLUSH();
116             return false;
117         }
118     }
119 
120     if (UART_test){
121         uart_packets_received++;
122     }
123     return true;
124 }
125 
126 static bool check_sequence_number(uint16_t received_seq)
127 {
128     if (first_packet){
129         first_packet = false;
130         last_seq = received_seq;
131         return true;
132     }
133     // Calculate how many sequence numbers were skipped
134     uint16_t delta = (uint16_t)(received_seq - last_seq);
135 
136     // Normal behavior
137     if (delta == 1){
138         last_seq = received_seq;
139         return true;
140     }
141 
142     // Plausible forward gap
143     if (delta > 1 && delta < MAX_PLAUSIBLE_SEQ_GAP){
```

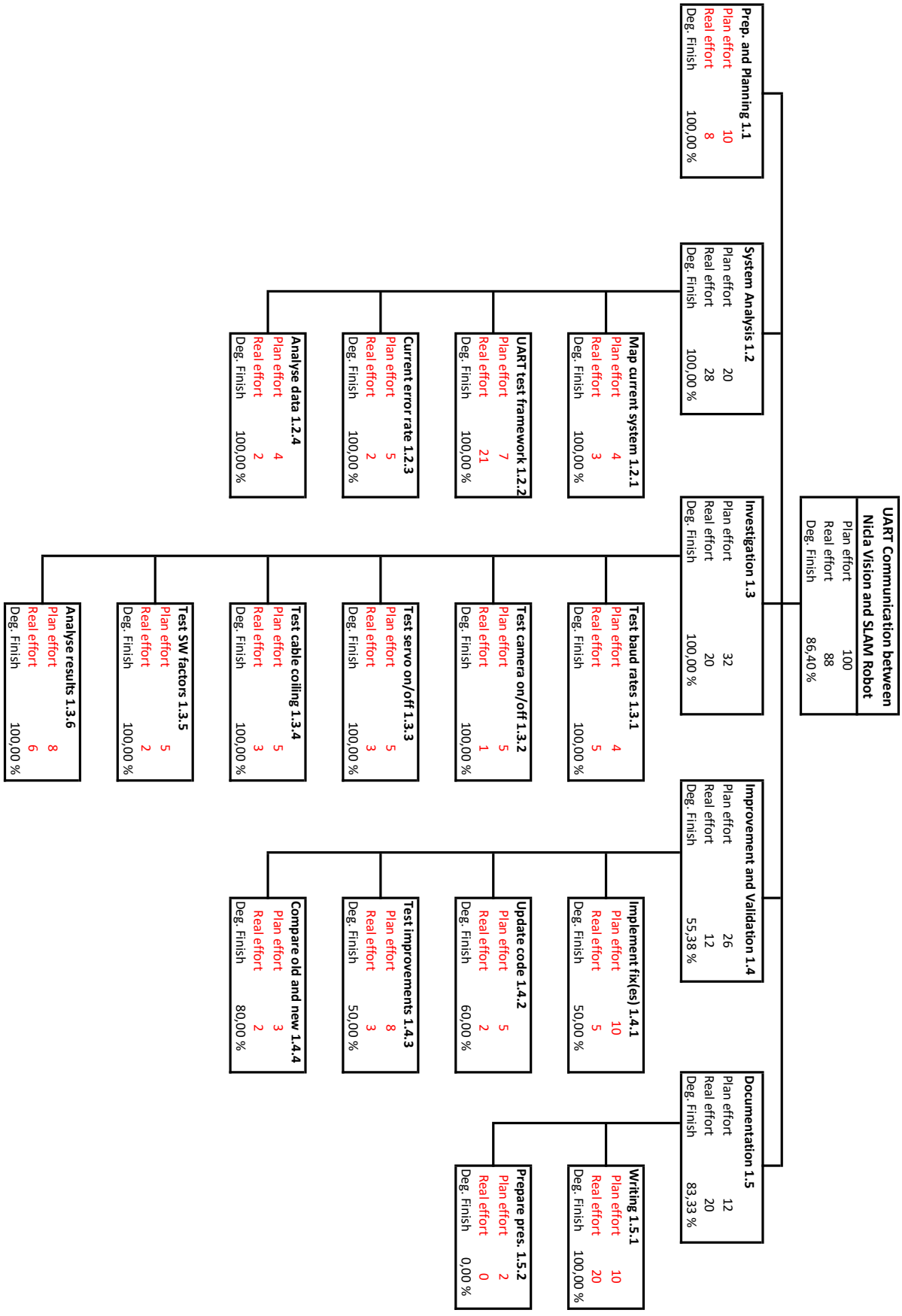
```
144         uint16_t missed = delta - 1;
145 
146         if (UART_test){
147             uart_missing_packets += missed;
148         }
149 
150         NRF_LOG_WARNING( Sequence gap: previous=%u received=%u
    missed=%u ,
151                         last_seq,received_seq,missed);
152 
153         last_seq = received_seq;
154         NRF_LOG_FLUSH();
155 
156         return true;
157     }
158 
159     // Duplicate, backwards, or implausibly large sequence jump
160     if (UART_test){
161         uart_suspicious_seq++;
162     }
163 
164     NRF_LOG_WARNING(
165          Suspicious sequence: previous=%u received=%u delta=%u ,
166         last_seq,
167         received_seq,
168         delta
169     );
170 
171     NRF_LOG_FLUSH();
172 
173     // No update in recived_seq
174     return false;
175 }
176 
177 CameraLineEstimate_t uarte1_get_line_estimate(void)
178 {
179     CameraLineEstimate_t result = {0};
180     uarte1_request_data();
181 
```

```
182     if (!uarte1_receive_data())
183     {
184         return result;
185     }
186 
187     result.seq = rx_buffer[0] | (rx_buffer[1] << 8);  // For test
188     result.x_start_mm = rx_buffer[2] | (rx_buffer[3] << 8);
189     result.x_width_mm = rx_buffer[4] | (rx_buffer[5] << 8);
190     result.distance_mm = rx_buffer[6] | (rx_buffer[7] << 8);
191 
192     // Reject suspicious sequence numbers before using measurements
193     if (!check_sequence_number(result.seq))
194     {
195         NRF_LOG_WARNING(
196              Rejecting suspicious packet seq=%u ,
197             result.seq
198         );
199         return (CameraLineEstimate_t){0};
200     }
201 
202     return result;
203 }
204 
205 static void send_camera_mqtt_update(CameraLineEstimate_t estimate)
206 {
207     mqttsn_camera_msg_t cameraMsg = {0};
208     cameraMsg.identifier = ID;
209     cameraMsg.start = estimate.x_start_mm;
210     cameraMsg.width = estimate.x_width_mm;
211     cameraMsg.distance = estimate.distance_mm;
212 
213     publish_camera( v2/robot/cam , cameraMsg, sizeof(cameraMsg), 0,
    0);
214 }
215 
216 static void uarte1_print_test_statistics(void)
217 {
218     uint32_t requested = uart_packets_requested;
219     uint32_t received = uart_packets_received;
```

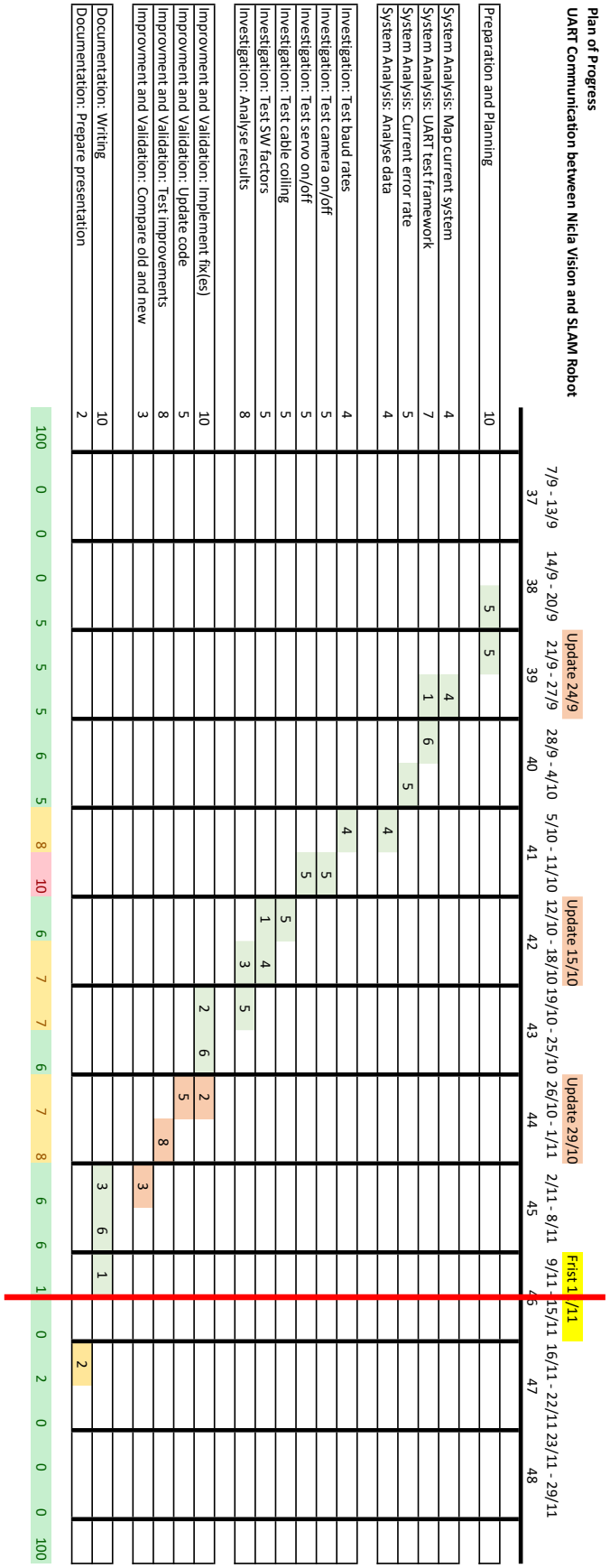
```
220     uint32_t timeouts = uart_timeouts;
221     uint32_t errors = uart_errors;
222     uint32_t missing = uart_missing_packets;
223     uint32_t suspicious = uart_suspicious_seq;
224     uint32_t lost = requested - received;
225 
226     NRF_LOG_INFO( ========== UART TEST ========== );
227     NRF_LOG_INFO( Packets requested: %d , requested);
228     NRF_LOG_INFO( Packets received:  %d , received);
229     NRF_LOG_INFO( Timeouts:          %d , timeouts);
230     NRF_LOG_INFO( UART errors:       %d , errors);
231     NRF_LOG_INFO( Missing packets    %d , missing);
232     NRF_LOG_INFO( Suspicious seq.    %d , suspicious);
233     NRF_LOG_INFO( Not received:      %d , lost);
234     NRF_LOG_INFO( ================================ );
235 
236     NRF_LOG_FLUSH();
237 }
238 
239 void task_camera(void *pvParameters)
240 {
241     TickType_t xLastWakeTime = xTaskGetTickCount();
242     CameraLineEstimate_t estimate;
243 
244     uarte1_init();  // Initialize UARTE peripheral
245 
246     // Wait until MQTT-SN is connected
247     while (!mqttsn_client_is_connected()) {
248         vTaskDelay(pdMS_TO_TICKS(100));
249     }
250 
251     NRF_LOG_INFO( Camera task started. );
252 
253     for (;;)
254     {
255         xLastWakeTime = xTaskGetTickCount();
256 
257         // Get line estimate from camera
258         estimate = uarte1_get_line_estimate();
```

```
259
260         // Publish via MQTT-SN
261         send_camera_mqtt_update(estimate);
262
263         // Debug logging
264         NRF_LOG_INFO( seq: %u, x_start_mm: %d, x_width_mm: %d,
    distance_mm: %d ,
265                       estimate.seq, estimate.x_start_mm, estimate.
    x_width_mm, estimate.distance_mm);
266         NRF_LOG_FLUSH();
267
268         // Print test progress every log interval
269         if (UART_test &&
270             uart_packets_requested > 0 &&
271             (uart_packets_requested % UART_TEST_LOG_INTERVAL) == 0)
272         {
273             uarte1_print_test_statistics();
274         }
275
276         // Finish test after processing the final request
277         if (UART_test &&
278             uart_packets_requested >= UART_TEST_REQUEST_LIMIT)
279         {
280             uarte1_print_test_statistics();
281
282             UART_test = false;
283
284             NRF_LOG_INFO( UART test finished after %lu requests ,
    uart_packets_requested);
285             NRF_LOG_FLUSH();
286         }
287
288         // Delay until next cycle
289         vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(200)); // Was
     400, 200 with test
290     }
291 }
```

D Work Breakdown Structure



E Plan of Progress



F AI Declaration

NTNU

Deklarasjon om KI-hjelpemidler

Har det i utarbeidingen av denne rapporten/avhandlingen blitt anvendt KI-baserte hjelpemidler?

Nei ✓ Ja

Hvis nei, signer nederst. Hvis ja: spesifser type av verktøy og bruksområde under.

Tekst

✓ **Stavekontroll.** Er deler av teksten kontrollert av: Grammarly, Ginger, Grammarbot, LanguageTool, ProWritingAid, Sapling, Trinka.ai eller lignende verktøy?

✓ **Tekstgenerering.** Er deler av teksten generert av: ChatGPT, GrammarlyGO, Copy.AI, WordAi, WriteSonic, Jasper, Simplified, Rytr eller lignende verktøy?

✓ **Skriveassistanse.** Er en eller flere av ideene eller fremgangsmåtene i oppgaven foreslått av: ChatGPT, Google Bard, Bing chat, YouChat eller lignende verktøy?

Hvis ja til anvendelse av et tekstverktøy - spesifiser bruken her:

ChatGPT/Overleafs KI ble hovedsakelig brukt til stavekontroll/reformulering og for å foreslå idéer til tekstutforming, altså ingen hele blokker er kun KI generert.

Koder og algoritmer

✓ **Programmeringsassistanse.** Er deler av koden/algoritmene som i) fremtrer direkte i rapporten eller ii) har blitt anvendt for produksjon av resultater slik som figurer, tabeller eller tallverdier blitt generert av: GitHub Copilot, CodeGPT, Google Codey/Studio Bot, Replit Ghostwriter, Amazon CodeWhisperer, GPT Engineer, ChatGPT, Google Bard eller lignende verktøy?

Hvis ja til anvendelse av et programmeringsverktøy - spesifiser bruken her:

Det er blitt brukt KI verktøy som ChatGPT til kodehjelp.

Bilder og figurer

Bildegenerering. Er ett eller flere av bildene/figurene i rapporten blitt generert av: Midjourney, Jasper, WriteSonic, Stability AI, Dall-E eller lignende verktøy?

Hvis ja til anvendelse av et bildeverktøy - spesifiser bruken her:

Andre KI-verktøy. Har andre typer av verktøy blitt anvendt? Hvis ja spesifiser bruken her:

✓ Jeg er kjent med NTNUs regelverk for bruk av kunstig intelligens. Jeg har redegjort for all anvendelse av kunstig intelligens enten i) direkte i rapporten eller ii) i dette skjemaet.

Martin G. Reitan/12.11.26/Trondheim

Underskrift/Dato/Sted